

GUPTASANDWITCH

Hotel Management System

Complete Project Documentation

Document Version: 1.0

Last Updated: May 20, 2026

Type: Multi-Outlet Restaurant Management System

Table of Contents

Table of Contents.....	2
1. Project Overview.....	4
Key Features.....	4
Business Purpose	4
2. Technology Stack	5
Current Implementation	5
Recommended Backend Stack	5
3. System Architecture.....	6
User Roles	6
Admin	6
Staff	6
Application Modules	6
Admin Panel	6
Staff Panel	6
4. Workflow Analysis.....	7
Order Processing Workflow	7
Detailed Workflow Steps	7
1. Order Creation (POS)	7
2. KOT Generation.....	7
3. Kitchen Processing	7
4. Payment Processing	7
5. Online Platform Orders (Swiggy/Zomato).....	7
Multi-Outlet Workflow	8
5. Database Schema.....	9
Key Table Definitions.....	10
Users Table	10
Menu Items Table	10
Orders Table.....	10
Payments Table	10
6. Table Relationships.....	12
API Structure.....	13
Authentication.....	13
User Management.....	13

Outlet Management	13
Menu Management.....	13
Order Management	14
KOT Management	14
Customer, Inventory & Platform.....	14
Reports.....	15
8. Security Considerations	16
Authentication & Authorization.....	16
Data Protection.....	16
Audit Logging	16
9. Future Enhancements.....	17
Phase 1 — Backend Development.....	17
Phase 2 — Advanced Features	17
Phase 3 — Analytics & AI.....	17
Phase 4 — Scalability.....	17

1. Project Overview

GuptaSandwich is a comprehensive restaurant management system designed for multi-location food outlets. The system covers end-to-end operations from point-of-sale transactions to kitchen management, online delivery integrations, and financial reporting.

Attribute	Details
Project Name	GuptaSandwich
Type	Multi-Outlet Restaurant Management System
Current State	React Frontend with Local State Management
Target State	Full-Stack Application with Backend API and Database
Document Version	1.0
Last Updated	May 20, 2026

Key Features

- Multi-Outlet Support: Manage multiple restaurant locations from a single admin panel
- Dual Pricing Structure: Different pricing for dine-in, parcel, and online platforms
- Real-Time Order Processing: Live order tracking from Swiggy and Zomato
- Kitchen Display System: KOT management for kitchen staff
- Comprehensive Reporting: Sales analytics, payment tracking, and GST reports
- Customer CRM: Track customer visits, orders, and preferences
- Inventory Management: Stock level monitoring and reorder alerts

Business Purpose

The system handles the following core business functions:

- Point of Sale (POS) operations
- Kitchen Order Tickets (KOT) management
- Online food delivery platform integration (Swiggy, Zomato)
- Multi-outlet management and consolidated reporting
- Customer relationship management and CRM tracking
- Inventory tracking and alerts
- Financial accounting and reporting

2. Technology Stack

Current Implementation

Component	Technology
Frontend Framework	React 19.2.6
Build Tool	Create React App (react-scripts 5.0.1)
Charting Library	Chart.js 4.5.1 / react-chartjs-2 5.3.1
State Management	React Hooks (useState, useEffect)
Authentication	Session-based (sessionStorage)
Styling	CSS Modules

Recommended Backend Stack

Component	Technology
Backend Framework	Node.js with Express.js
Database	PostgreSQL (recommended) or MongoDB
ORM	Prisma (for PostgreSQL) or Mongoose (for MongoDB)
Authentication	JWT (JSON Web Tokens)
API Documentation	Swagger / OpenAPI
Real-Time Communication	Socket.io (live orders, KOT updates)
File Storage	AWS S3 or Cloudinary (receipts, bills)

3. System Architecture

User Roles

Admin

- Full system access
- Outlet management
- Menu and pricing control
- Financial reports and accounting
- User management

Staff

- POS operations
- KOT management
- Order processing
- Customer interaction
- Basic reporting

Application Modules

Admin Panel

- Dashboard: Outlet performance analytics with charts
- Dishes Management: Menu item CRUD with multi-pricing
- Reports: Daily sales, payment modes, GST, category-wise, hourly, platform-wise
- Accounting: Financial ledger with bill uploads
- Outlets: Multi-location management with credentials

Staff Panel

- POS: Order taking with dine-in/parcel toggle
- KOT: Kitchen display system with item tracking
- Live Orders: Swiggy/Zomato integration
- Reports: Sales analytics and order history
- Customers: Customer management and order history
- Inventory: Stock level monitoring

4. Workflow Analysis

Order Processing Workflow

The end-to-end order lifecycle follows these stages:

Customer Order	POS System	Order Creation	KOT Generation	Kitchen Prep	Order Completion	Payment	Receipt	Done
----------------	------------	----------------	----------------	--------------	------------------	---------	---------	------

Detailed Workflow Steps

1. Order Creation (POS)

- Staff selects dine-in or parcel mode
- Items added from menu with appropriate pricing
- Discounts applied (percentage or flat)
- Order subtotal calculated with GST (5%)
- Table assigned for dine-in orders

2. KOT Generation

- Kitchen Order Ticket created from order
- Items sent to kitchen display system
- Preparation time estimated per item
- Urgent orders flagged for priority

3. Kitchen Processing

- Kitchen staff receive and acknowledge KOT
- Individual items marked as ready
- Partial dispatching allowed for large orders
- Full order dispatch when all items complete

4. Payment Processing

- Payment method selected (Cash / Online / UPI / Card)
- Payment recorded in system
- Receipt generated and optionally printed
- Order marked as completed

5. Online Platform Orders (Swiggy/Zomato)

- Orders received via API integration
- Auto-accept or manual accept options
- Status updates sent to platform in real time
- Delivery tracking managed through platform

Multi-Outlet Workflow

- Central admin manages all outlets from single panel
- Each outlet has independent operational flow
- Consolidated reporting across all outlets
- Outlet-specific pricing and menu availability

5. Database Schema

The system uses 20 tables organized across core functional areas. Below is a summary of each table's purpose, followed by key schema definitions.

Table Name	Purpose
users	System users — Admin and Staff accounts
outlets	Restaurant outlet/location information
categories	Menu item categories
menu_items	Menu items with multi-tier pricing
menu_item_outlets	Junction: menu items available at each outlet
ingredients	Ingredient master list
menu_item_ingredients	Junction: ingredients per menu item
tables	Dining table information per outlet
customers	Customer CRM data
orders	Customer orders (all types)
order_items	Line items within an order
kots	Kitchen Order Tickets
kot_items	Items within a KOT
payments	Payment transactions
inventory_categories	Inventory classification categories
inventory_items	Inventory stock levels per outlet
accounting_entries	Financial ledger entries
platform_orders	Swiggy/Zomato platform orders
discounts	Discount configuration rules
reports	Cached/generated report data

Key Table Definitions

Users Table

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role ENUM('admin', 'staff') NOT NULL,
```

```

    name          VARCHAR(255) NOT NULL,
    phone          VARCHAR(20),
    outlet_id      INTEGER REFERENCES outlets(id),
    is_active      BOOLEAN DEFAULT true,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Menu Items Table

```

CREATE TABLE menu_items (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(255) NOT NULL,
    category_id     INTEGER REFERENCES categories(id),
    description      TEXT,
    is_veg          BOOLEAN DEFAULT true,
    dine_in_price   DECIMAL(10,2) NOT NULL,
    parcel_price    DECIMAL(10,2) NOT NULL,
    swiggy_price    DECIMAL(10,2),
    zomato_price    DECIMAL(10,2),
    available_swiggy BOOLEAN DEFAULT true,
    available_zomato BOOLEAN DEFAULT true,
    is_active       BOOLEAN DEFAULT true,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Orders Table

```

CREATE TABLE orders (
    id              SERIAL PRIMARY KEY,
    order_number    VARCHAR(50) UNIQUE NOT NULL,
    outlet_id       INTEGER REFERENCES outlets(id),
    table_id        INTEGER REFERENCES tables(id),
    customer_id     INTEGER REFERENCES customers(id),
    order_type      ENUM('dine_in','parcel','swiggy','zomato') NOT NULL,
    status          ENUM('pending','preparing','ready','completed','cancelled'),
    subtotal        DECIMAL(10,2) NOT NULL,
    discount_amount DECIMAL(10,2) DEFAULT 0,
    gst_amount      DECIMAL(10,2) NOT NULL,
    total_amount    DECIMAL(10,2) NOT NULL,
    created_by      INTEGER REFERENCES users(id),
    order_time      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Payments Table

```

CREATE TABLE payments (
    id              SERIAL PRIMARY KEY,
    order_id        INTEGER REFERENCES orders(id),
    payment_method  ENUM('cash','upi','card','wallet') NOT NULL,
    amount          DECIMAL(10,2) NOT NULL,
    transaction_id  VARCHAR(100),
    status          ENUM('pending','completed','failed','refunded'),
    payment_time    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

);

6. Table Relationships

Relationship	Description
Users → Outlets (Many-to-One)	Staff belong to a specific outlet; Admins manage all outlets
Outlets → Tables (One-to-Many)	Each outlet has multiple dining tables
Outlets → Orders (One-to-Many)	Orders are placed at specific outlets
Outlets → InventoryItems (One-to-Many)	Inventory is tracked per outlet
Categories → MenuItems (One-to-Many)	Each menu item belongs to one category
MenuItems ↔ Outlets (Many-to-Many)	Items can be available at multiple outlets (via junction table)
MenuItems ↔ Ingredients (Many-to-Many)	Items composed of multiple ingredients (via junction table)
Orders → OrderItems (One-to-Many)	Each order contains multiple line items
Orders → KOTs (One-to-One)	Each order generates one Kitchen Order Ticket
Orders → Payments (One-to-Many)	Orders can have multiple payments (split, partial)
KOTs → KOTItems (One-to-Many)	Each KOT contains individual item entries
Customers → Orders (One-to-Many)	Customer CRM linked to order history
Tables → Orders (One-to-Many)	Dine-in orders assigned to specific tables

7. API Structure

RESTful API endpoints organized by functional domain. All endpoints are prefixed with /api.

Authentication

Method	Endpoint
POST	/api/auth/login
POST	/api/auth/logout
POST	/api/auth/refresh
GET	/api/auth/me

User Management

Method	Endpoint
GET	/api/users
POST	/api/users
GET	/api/users/:id
PUT	/api/users/:id
DELETE	/api/users/:id

Outlet Management

Method	Endpoint
GET	/api/outlets
POST	/api/outlets
GET	/api/outlets/:id
PUT	/api/outlets/:id
DELETE	/api/outlets/:id
PUT	/api/outlets/:id/status

Menu Management

Method	Endpoint
GET	/api/categories
POST	/api/categories

Method	Endpoint
GET	/api/menu-items
POST	/api/menu-items
GET	/api/menu-items/:id
PUT	/api/menu-items/:id
DELETE	/api/menu-items/:id
GET	/api/menu-items/:id/ingredients

Order Management

Method	Endpoint
GET	/api/orders
POST	/api/orders
GET	/api/orders/:id
PUT	/api/orders/:id
DELETE	/api/orders/:id
POST	/api/orders/:id/items
PUT	/api/orders/:id/items/:itemId
DELETE	/api/orders/:id/items/:itemId
POST	/api/orders/:id/payment
POST	/api/orders/:id/discount

KOT Management

Method	Endpoint
GET	/api/kots
POST	/api/kots
GET	/api/kots/:id
PUT	/api/kots/:id/status
PUT	/api/kots/:id/items/:itemId/status
POST	/api/kots/:id/dispatch

Customer, Inventory & Platform

Reports

Method	Endpoint
GET	/api/reports/daily-sales
GET	/api/reports/payment-modes
GET	/api/reports/gst
GET	/api/reports/category-wise
GET	/api/reports/hourly
GET	/api/reports/platform-wise
GET	/api/reports/food-cost

8. Security Considerations

Authentication & Authorization

- Implement JWT-based authentication with refresh tokens
- Role-based access control (RBAC) for Admin and Staff separation
- Password hashing using bcrypt with appropriate salt rounds
- Session management with short-lived access tokens
- API rate limiting to prevent brute-force attacks

Data Protection

- Encrypt sensitive data (passwords, customer PII)
- Enforce HTTPS for all API communications
- Implement strict CORS policies
- Use parameterized queries to prevent SQL injection
- Apply XSS protection headers and input sanitization

Audit Logging

- Log all user actions with timestamps and actor IDs
- Track order modifications and cancellations
- Monitor payment transactions for anomalies
- Record all login attempts (successful and failed)

9. Future Enhancements

Phase 1 — Backend Development

- Set up Node.js/Express backend with modular routing
- Implement PostgreSQL database with Prisma ORM
- Create all RESTful API endpoints
- Implement JWT authentication and RBAC middleware
- Set up Socket.io for real-time KOT and order updates

Phase 2 — Advanced Features

- Swiggy/Zomato API integration for live order ingestion
- Real-time KOT updates via WebSockets
- Email and SMS notifications for order status
- Advanced reporting with CSV/PDF export options
- Mobile app development (React Native)

Phase 3 — Analytics & AI

- Predictive inventory management based on sales trends
- Customer behavior analytics and segmentation
- Sales forecasting with ML models
- Automated reorder suggestions
- Loyalty program integration

Phase 4 — Scalability

- Microservices architecture for independent scaling
- Load balancing and horizontal scaling
- Caching layer using Redis for high-traffic endpoints
- CDN implementation for static assets
- Multi-region deployment for low latency

GuptaSandwich Hotel Management System

Complete Project Documentation | Version 1.0 | May 2026